

TypedMemory Shapes, Paths, And Union Representations

A short design note capturing the path we explored: records and sealed interfaces as the type model, memory regions as the representation model, and MemShape/RegionPath as the bridge between them.

1. The Core Split

The important separation is not between Java and native memory. It is between type, region, and interpretation. Once these are separate, many low-level C/OpenCL layout idioms become expressible without giving up the Java record model.

```
type model      -> records, sealed interfaces, variants
memory region   -> byte offset, layout, array coordinates
interpretation  -> path, shape, tag rule, overlay rule
```

RegionPath answers: where is the memory region? MemShape answers: how should union choices under that region be interpreted?

2. RegionPath: Structural Lensing

A RegionPath is a validated structural key. If two separately built paths describe the same root, tokens, and leaf, they compare equal and can be used as cache keys.

```
record Pixel(int i, int j) {}
record Point(float x, float y, Pixel p) {}

var p1 = MemPaths.from(Point.class)
    .field("p", Pixel.class)
    .region();

var p2 = MemPaths.from(Point.class)
    .field("p", Pixel.class)
    .region();

assert p1.equals(p2);
```

Arrays add coordinates to the path. Fixed indexes close the coordinate. Open indexes become handle coordinates later.

```
record Palette(@size(4) Pixel[] pixels) {}

var fixed = MemPaths.from(Palette.class)
    .array("pixels", Pixel.class)
    .at(2)
    .region();

var open = MemPaths.from(Palette.class)
    .array("pixels", Pixel.class)
    .any()
    .region();
```

3. MemShape: Concrete Union Interpretation

MemShape describes which concrete variants are used under a record or union root. It validates completeness while building, so missing union choices are caught before generated handles are involved.

```
record DeviceRoot(Device device) {}
sealed interface Device permits SmartDevice, SimpleDevice {}
record SmartDevice(Profile profile) implements Device {}
record SimpleDevice(int value) implements Device {}

var shape = MemShapes.of(DeviceRoot.class)
    .union("device", Device.class, device -> device
        .variant(SmartDevice.class, smart -> smart
            .field("profile", Profile.class, profile -> ...)))
    .shape();
```

A shape can also start at a union root when the region being decoded is itself declared as a sealed interface.

```
var shape = MemShapes.union(Device.class)
    .variant(SmartDevice.class, smart -> smart
        .field("profile", Profile.class, profile -> ...))
    .shape();
```

4. Sibling Union Fields

A record can contain multiple independent union fields. MemShape is not a single chain; it stores sibling branches by field name.

```
record Root(Left left, Right right) {}

var shape = MemShapes.of(Root.class)
    .union("left", Left.class, left -> left
        .variant(A.class, a -> a
            .union("child", AChild.class, child -> child
                .variant(AChild1.class))))
    .union("right", Right.class, right -> right
        .variant(D.class, d -> d
            .union("mode", Mode.class, mode -> mode
                .variant(ModeX.class))))
    .shape();
```

5. Tagged Union: Sibling Tag + Payload

This matches a common systems layout: a record owns a discriminator field and a union payload field. The tag is outside the payload but in the same record.

```
sealed interface LightPayload permits AreaLight, IblLight, SpotLight {}

record Light(int type, LightPayload payload, float multiplier) {}
record AreaLight(int id, int shapeidx, int primidx, int padding) implements LightPayload {}
record IblLight(int tex, int texReflection, int texRefraction, int texTransparency) implements LightPayload {}
record SpotLight(float ia, float oa, float f, int padding) implements LightPayload {}

var shape = MemShapes.of(Light.class)
    .taggedUnion("type", "payload", LightPayload.class, light -> light
        .caseOf(1, AreaLight.class)
        .caseOf(2, IblLight.class)
        .caseOf(3, SpotLight.class))
    .shape();
```

6. Tagged Union Array: Per-Element Tags

If every array element can have a different active variant, the array element should be a tagged slot. A single uniform shape is not enough for mixed elements.

```
record TaggedBatch(TaggedDevice[] devices) {}
record TaggedDevice(int tag, Device payload) {}

var shape = MemShapes.of(TaggedBatch.class)
    .taggedUnionArray(
        "devices",
        TaggedDevice.class,
        "tag",
        "payload",
        Device.class,
        devices -> devices
            .caseOf(1, SmartDevice.class)
            .caseOf(2, SimpleDevice.class))
    .shape();
```

Supported tag carriers are byte, short, int, long, boolean, and char. Floating-point tags are intentionally excluded.

7. Overlay Union: C-Style Shared Storage

Some C/OpenCL unions are not tag + payload. They are one storage region with several overlay views. The InputMapData example is this form: the tag is found by viewing the same bytes as one particular variant.

```
// OpenCL shape:
typedef struct _InputMapData {
    union {
        struct { float3 value; } float_value;
        struct { int idx; int placeholder[2]; int type; } int_values;
    };
} InputMapData;

sealed interface InputMapUnion permits FloatValue, IntValues {}

record InputMapData(InputMapUnion u) {}
record Float3(float x, float y, float z) {}
record FloatValue(Float3 value) implements InputMapUnion {}
record IntValues(int idx, int placeholder0, int placeholder1, int type) implements InputMapUnion {}

var shape = MemShapes.of(InputMapData.class)
    .union("u", InputMapUnion.class, u -> u
        .overlay()
        .tagFrom(IntValues.class, "type")
        .caseOf(0, FloatValue.class)
        .caseOf(2, IntValues.class))
    .shape();
```

tagFrom(IntValues.class, "type") means: interpret the union bytes as IntValues, then read the type field.

8. How The Pieces Compose

The future handle/view layer can compose a RegionPath and a MemShape. The path locates the bytes. The shape says how to realize sealed-interface values and nested variants.

```
Device d = mem.handle(devicePath)
    .view(deviceShape)
    .get(rootIndex, deviceIndex);
```

For shaped reads, tag checks should be part of get(). A lower-level matches() can exist, but pattern matching and shaped decoding should be the nice public path.

9. Coverage Summary

Case	Shape API	Representation idea
Static union branch	<code>.union(...).variant(...)</code>	One known concrete variant
Union root	<code>MemShapes.union(...)</code>	Shape starts at a sealed interface
Sibling tag	<code>.taggedUnion(tag, payload, ...)</code>	One record owns tag and payload fields
Tagged array slots	<code>.taggedUnionArray(...)</code>	Each element owns tag and payload fields
Overlay-derived tag	<code>.union(...).overlay().tagFrom(...)</code>	C union bytes, tag read through one variant layout
Region focus	<code>RegionPath / MemPaths</code>	Where the bytes are

This does not claim to cover every C layout trick forever. It does cover the major practical families found in record/sealed type models, GPU payload layouts, tag + payload slots, mixed union arrays, and overlay-derived discriminators.